Fleury's Algorithm for printing Eulerian Path or Circuit

Eulerian Path is a path in graph that visits every edge exactly once. Eulerian Circuit is an Eulerian Path which starts and ends on the same vertex.

We strongly recommend to first read the following post on Euler Path and Circuit.
http://www.geeksforgeeks.org/eulerian-path-and-circuit/

In the above mentioned post, we discussed the problem of finding out whether a given graph is Eulerian or not. In this post, an algorithm to print Eulerian trail or circuit is discussed.

Following is Fleury's Algorithm for printing Eulerian trail or cycle (Source Ref1).

1. Make sure the graph has either 0 or 2 odd vertices.

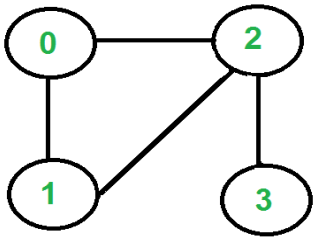
2. If there are 0 odd vertices, start anywhere. If there are 2 odd vertices, start at one of them.

3. Follow edges one at a time. If you have a choice between a bridge and a non-bridge, *always choose*

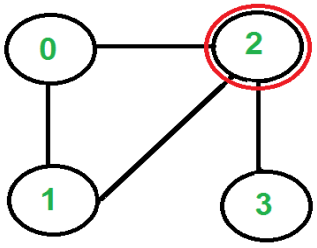
the non-bridge.

4. Stop when you run out of edges.

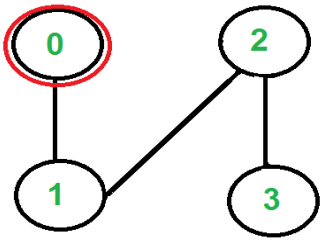
The idea is, ***"don't burn [bridges]"*** so that we can come back to a vertex and traverse remaining edges. For example let us consider the following graph.



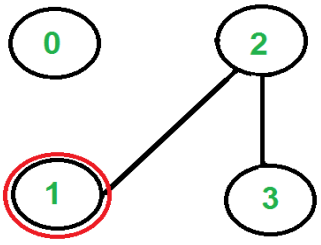
There are two vertices with odd degree, '2' and '3', we can start path from any of them. Let us start tour from vertex '2'.



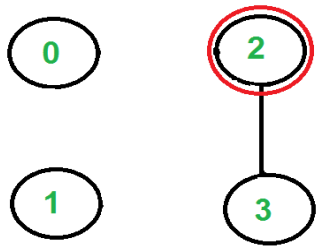
There are three edges going out from vertex '2', which one to pick? We don't pick the edge '2-3′ because that is a bridge (we won't be able to come back to '3'). We can pick any of the remaining two edge. Let us say we pick '2-0′. We remove this edge and move to vertex '0'.



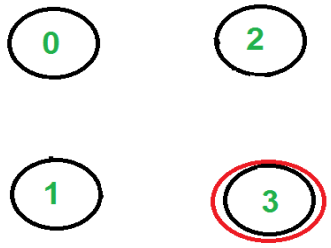
There is only one edge from vertex '0', so we pick it, remove it and move to vertex '1'. Euler tour becomes '2-0 0-1′.



There is only one edge from vertex '1', so we pick it, remove it and move to vertex '2'. Euler tour becomes '2-0 0-1 1-2′



Again there is only one edge from vertex 2, so we pick it, remove it and move to vertex 3. Euler tour becomes ‘2-0 0-1 1-2 2-3′



There are no more edges left, so we stop here. Final tour is ‘2-0 0-1 1-2 2-3′.

See this for and this fore more examples.

Following is C++ implementation of above algorithm. In the following code, it is assumed that the given graph has an Eulerian trail or Circuit. The main focus is to print an Eulerian trail or circuit. We can use isEulerian() to first check whether there is an Eulerian Trail or Circuit in the given graph.

We first find the starting point which must be an odd vertex (if there are odd vertices) and store it in variable ‘u’. If there are zero odd vertices, we start from vertex ‘0’. We call printEulerUtil() to print Euler tour starting with u. We traverse all adjacent vertices of u, if there is only one adjacent vertex, we immediately consider it. If there are more than one adjacent vertices, we consider an adjacent v only if edge u-v is not a bridge. How to find if a given is edge is bridge? We count number of vertices reachable from u. We remove edge u-v and again count number of reachable vertices from u. If number of reachable vertices are reduced, then edge u-v is a bridge. To count reachable vertices, we can either use BFS or DFS, we have used DFS in the above code. The function DFSCount(u) returns number of vertices reachable from u.
Once an edge is processed (included in Euler tour), we remove it from the graph. To remove the edge, we replace the vertex entry with -1 in adjacency list. Note that simply deleting the node may not work as the code is recursive and a parent call may be in middle of adjacency list.

```
// A C++ program print Eulerian Trail in a given Eulerian or Semi-Eulerian
#include <iostream>
#include <string.h>
#include <algorithm>
#include <list>
using namespace std;

// A class that represents an undirected graph
class Graph
{
  int V;    // No. of vertices
  list<int> *adj;    // A dynamic array of adjacency lists
public:
    // Constructor and destructor
  Graph(int V)  { this->V = V;  adj = new list<int>[V];  }
  ~Graph()      { delete [] adj;  }
```

```
    // functions to add and remove edge
    void addEdge(int u, int v) {  adj[u].push_back(v); adj[v].push_back(u); }
    void rmvEdge(int u, int v);

    // Methods to print Eulerian tour
    void printEulerTour();
    void printEulerUtil(int s);

    // This function returns count of vertices reachable from v. It does DFS
    int DFSCount(int v, bool visited[]);

    // Utility function to check if edge u-v is a valid next edge in
    // Eulerian trail or circuit
    bool isValidNextEdge(int u, int v);
};

/* The main function that print Eulerian Trail. It first finds an odd
   degree vertex (if there is any) and then calls printEulerUtil()
   to print the path */
void Graph::printEulerTour()
{
    // Find a vertex with odd degree
    int u = 0;
    for (int i = 0; i < V; i++)
        if (adj[i].size() & 1)
          {   u = i; break;  }

    // Print tour starting from oddv
    printEulerUtil(u);
    cout << endl;
}

// Print Euler tour starting from vertex u
void Graph::printEulerUtil(int u)
{
    // Recur for all the vertices adjacent to this vertex
    list<int>::iterator i;
    for (i = adj[u].begin(); i != adj[u].end(); ++i)
    {
        int v = *i;

        // If edge u-v is not removed and it's a a valid next edge
        if (v != -1 && isValidNextEdge(u, v))
        {
            cout << u << "-" << v << "  ";
            rmvEdge(u, v);
            printEulerUtil(v);
        }
    }
}

// The function to check if edge u-v can be considered as next edge in
// Euler Tout
bool Graph::isValidNextEdge(int u, int v)
{
    // The edge u-v is valid in one of the following two cases:
```

```
  // 1) If v is the only adjacent vertex of u
  int count = 0;  // To store count of adjacent vertices
  list<int>::iterator i;
  for (i = adj[u].begin(); i != adj[u].end(); ++i)
     if (*i != -1)
        count++;
  if (count == 1)
    return true;


  // 2) If there are multiple adjacents, then u-v is not a bridge
  // Do following steps to check if u-v is a bridge

  // 2.a) count of vertices reachable from u
  bool visited[V];
  memset(visited, false, V);
  int count1 = DFSCount(u, visited);

  // 2.b) Remove edge (u, v) and after removing the edge, count
  // vertices reachable from u
  rmvEdge(u, v);
  memset(visited, false, V);
  int count2 = DFSCount(u, visited);

  // 2.c) Add the edge back to the graph
  addEdge(u, v);

  // 2.d) If count1 is greater, then edge (u, v) is a bridge
  return (count1 > count2)? false: true;
}

// This function removes edge u-v from graph.  It removes the edge by
// replacing adjcent vertex value with -1.
void Graph::rmvEdge(int u, int v)
{
  // Find v in adjacency list of u and replace it with -1
  list<int>::iterator iv = find(adj[u].begin(), adj[u].end(), v);
  *iv = -1;

  // Find u in adjacency list of v and replace it with -1
  list<int>::iterator iu = find(adj[v].begin(), adj[v].end(), u);
  *iu = -1;
}

// A DFS based function to count reachable vertices from v
int Graph::DFSCount(int v, bool visited[])
{
  // Mark the current node as visited
  visited[v] = true;
  int count = 1;

  // Recur for all vertices adjacent to this vertex
  list<int>::iterator i;
  for (i = adj[v].begin(); i != adj[v].end(); ++i)
      if (*i != -1 && !visited[*i])
          count += DFSCount(*i, visited);
```

```
  return count;
}

// Driver program to test above function
int main()
{
  // Let us first create and test graphs shown in above figure
  Graph g1(4);
  g1.addEdge(0, 1);
  g1.addEdge(0, 2);
  g1.addEdge(1, 2);
  g1.addEdge(2, 3);
  g1.printEulerTour();

  Graph g2(3);
  g2.addEdge(0, 1);
  g2.addEdge(1, 2);
  g2.addEdge(2, 0);
  g2.printEulerTour();

  Graph g3(5);
  g3.addEdge(1, 0);
  g3.addEdge(0, 2);
  g3.addEdge(2, 1);
  g3.addEdge(0, 3);
  g3.addEdge(3, 4);
  g3.addEdge(3, 2);
  g3.addEdge(3, 1);
  g3.addEdge(2, 4);
  g3.printEulerTour();

  return 0;
}
```

Output:

```
2-0  0-1  1-2  2-3
0-1  1-2  2-0
0-1  1-2  2-0  0-3  3-4  4-2  2-3  3-1
```

Note that the above code modifies given graph, we can create a copy of graph if we don't want the given graph to be modified.

Time Complexity: Time complexity of the above implementation is O $((V+E)^2)$. The function printEulerUtil() is like DFS and it calls isValidNextEdge() which also does DFS two times. Time complexity of DFS for adjacency list representation is O(V+E). Therefore overall time complexity is O((V+E)*(V+E)) which can be written as O(E^2) for a connected graph.

There are better algorithms to print Euler tour, we will soon be covering them as separate posts.

References:
http://www.math.ku.edu/~jmartin/courses/math105-F11/Lectures/chapter5-part2.pdf
http://en.wikipedia.org/wiki/Eulerian_path#Fleury.27s_algorithm

Please write comments if you find anything incorrect, or you want to share more information about

the topic discussed above

Related Topics:

- Karger's algorithm for Minimum Cut | Set 1 (Introduction and Implementation)
- Greedy Algorithms | Set 9 (Boruvka's algorithm)
- Assign directions to edges so that the directed graph remains acyclic
- K Centers Problem | Set 1 (Greedy Approximate Algorithm)
- Find the minimum cost to reach destination using a train
- Applications of Breadth First Traversal
- Optimal read list for given number of days
- Print all paths from a given source to a destination

Tags: Graph

Writing code in comment? Please use **ideone.com** and share the link here.

MJW ➚ Kenneth • 5 months ago

There's a simple algorithm form GraphMagics.com which I believe solves the problem:

1. Start with an empty stack and an empty circuit (eulerian path).

- If all vertices have even degree - choose any of them.

- If there are exactly 2 vertices having an odd degree - choose one of them.

- Otherwise no euler circuit or path exists.

2. If current vertex has no neighbors - add it to
circuit, remove the last vertex from the stack and set it as the current one. Otherwise
(in case it has neighbors) - add the vertex to the
stack, take any of its neighbors, remove the edge between selected neighbor and that
vertex, and set that neighbor as the current vertex.

Repeat step 2 until the current vertex has no more neighbors and the stack is empty.

▲ | ▼ • Reply • Share ›

MJW ➚ MJW • 5 months ago

Sorry about the poor formatting. I forgot to edit out the newlines from my cut-and-paste.

▲ | ▼ • Reply • Share ›

Anurag Singh • 9 months ago

The Fleury’s Algorithm is nicely implemented here. Ideas to handle edge removal, addition and bridge check on the fly for a given edge are really nice.

I just thought of two possible improvements (higher speed, at cost of more space):
1. In adjacency list representation, we always use a list and this data structure works very well in most of the cases. Because most of the time, we just do a linear traversal on the all nodes adjacent to a particular node. Here we have a bit different case. We are doing linear traversal as we normally do, but along with that, we are also searching a node v, in adjacency list of u, and then making v to -1 (interpreting that as if edge u -- v is deleted. The list data-structure can be replaced by a " Doubly Linked List + HashSet". While adding a new node, insert node at the end of list and store node's "ADDRESS" in the HashSet. This will make the search faster (constant if HashSet is good - get the ADDRESS of node being searched from HashSet and read value at that address from Linked List). While removing a node, search the node and delete it from both list and hashset. This is similar to LRU implementation idea. This way, we don't need "-1" check also that we are doing in above code.
2. To check whether a given edge is bridge or not, we are doing two time DFS. We may just use "disc" and "low" array (Tarjan's Bridge-finding algorithm) computation method (http://www.geeksforgeeks.org/b... This will require one DFS (Even we can stop ongoing

DFS as soon as we find that edge under consideration is a bridge)

Java: http://ideone.com/g2J6GM
Will get C++ code later sometime..

• Reply • Share ›

Ash • 2 years ago

Ah, I see now why my previously mentioned solution would not work. I came across a counter example.

• Reply • Share ›

Ash • 2 years ago

Would it be possible to avoid the DFS by instead checking the in-degree of the next vertex? If the next vertex only has in-degree of 1, then you are burning a bridge. My logic must be wrong here though because this would imply a linear-time algorithm and Wikipedia doesn't mention any existence of a linear time algorithm.

• Reply • Share ›

Susnata Roy Ash • a year ago

having an in-degree of 1 is not the only condition when its a bridge, when u remove a bridge number of connected components should increase.

http://s30.postimg.org/4fyghwb... - its a bridge, even though degree is not 1 for either of the vertices.

1 • Reply • Share ›

-
-
  - Interview Experiences
  - Advanced Data Structures
  - Dynamic Programming
  - Greedy Algorithms
  - Backtracking
  - Pattern Searching
  - Divide & Conquer
  - Mathematical Algorithms
  - Recursion
  - Geometric Algorithms

- # Popular Posts

  - All permutations of a given string
  - Memory Layout of C Programs
  - Understanding “extern” keyword in C
  - Median of two sorted arrays
  - Tree traversal without recursion and without stack!
  - Structure Member Alignment, Padding and Data Packing
  - Intersection point of two Linked Lists
  - Lowest Common Ancestor in a BST.
  - Check if a binary tree is BST or not
  - Sorted Linked List to Balanced BST

-

- # Recent Comments

  - Priyanshu Sharma

return ((n<<2) +(n<<1) + n)>>3;

Calculate 7n/8 without using division and multiplication operators · 14 minutes ago

- acodebreaker

  It redirects to Piyush kapoor's comment on this...

  Count total set bits in all numbers from 1 to n · 16 minutes ago

- Guest

  Another similar solution to this post could be...

  Online algorithm for checking palindrome in a stream · 21 minutes ago

- Praveen K. Dara

  nice algo to check either MajorityElement is...

  Find the Number Occurring Odd Number of Times · 36 minutes ago

- Mehboob_Elahi

  reverse a DLL . struct node *rev(struct node...

  Reverse a Doubly Linked List · 49 minutes ago

- Baba

  hey dude you are returning minimum among passed...

  How to Count Variable Numbers of Arguments in C? · 51 minutes ago